
Compute Over the Graph, Do Not Show It

What an execution graph is worth to a self-repairing agent, measured against injected faults

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 An execution graph records what a generated pipeline did rather than what a model
2 said it did, and is increasingly proposed as the substrate an agent uses to repair it-
3 self. We ask what that substrate is worth by injecting faults whose location is
4 known by construction, so trust decisions can be scored instead of trusted. The
5 graph pays off through computation over its structure, not through presentation to
6 a model. Computing over it works: a selector walking the dataflow to the causal
7 root identifies the faulty boundary in 21 of 21 contested cases where a source-
8 order baseline manages none, and its advantage is exactly the extent to which
9 declaration order diverges from execution order. Showing it to a model is more
10 delicate than expected: a context region selected in advance is indistinguishable
11 from a size-matched random one, yet a protocol that withholds nested detail and
12 lets the reviewer request it one step at a time matches sending everything on less
13 context and does separate from random, so the benefit lies in the interaction rather
14 than the slice. The sharpest evidence is where checks are placed. Over 90 data
15 faults, hand-written contracts detect 26 and localise *none*, firing every time on an
16 aggregating stage downstream of the cause, which is the downstream-blame bias
17 the model judges showed reproduced in a signal containing no model. Checks
18 learned from caught faults and placed where they entered detect 60 and localise
19 all 60, with zero false suspicion on held-out healthy runs once vocabulary checks
20 are restricted to domains the healthy population shows to be closed. Five mea-
21 surement artifacts caught during this work, each of which would have published a
22 confident wrong number, argue that such a benchmark is worth as much for what
23 it invalidates as for what it reports.

1 Introduction

25 An agent that works for a long time on one objective will eventually get something wrong, and what
26 happens next is a trust decision. It has to say which earlier steps still stand, which are contaminated,
27 and where to resume. Wrong in one direction it redoes sound work. Wrong in the other it builds on
28 a spoiled result, which is worse and much harder to see.

29 The architectural answer under study is the execution graph. A scanner captures the artifacts a
30 generated pipeline produced and the functions that produced them, and that record becomes the
31 durable state the agent reasons over. The appeal is that a graph built from execution cannot be talked
32 out of what happened. Recent work treats provenance of this kind as the substrate for agent trust
33 [11] and as the basis for recoverable execution from recorded checkpoints [13].

34 We take the architecture seriously enough to ask what it is worth, and the answer we find is narrower
35 and more useful than the one usually assumed. **The graph pays off through computation over its**

36 **structure, and not through presentation to a model.** Those are different uses of the same artifact
37 and they do not succeed together. A selector that walks the recorded dataflow to the causal root of
38 a set of flagged boundaries is exact where a baseline without that relation is useless. Acceptance
39 criteria attached to boundaries catch faults a model reading the same evidence does not. But handing
40 a model a graph-selected region of context, which is the use the substrate is most often justified by,
41 cannot be distinguished from handing it a size-matched random region, and accumulating history
42 costs two orders of magnitude more context for no gain.

43 Answering the question at all required a measurement the system does not have. Trust labels here
44 are produced by one model reviewing another, so nothing in the loop holds an independent answer;
45 when a reviewer calls a boundary trusted, no artifact can contradict it. There is good reason not to
46 take that on faith, since judges prefer text they find familiar [12] and a judge’s agreement with itself
47 can be high while its agreement with the truth is not [7], and a verifier checking only what it can
48 express admits confident false positives [5]. We supply the missing answer by injecting a fault into a
49 named boundary through the same scope machinery a repair must pass, so the poisoned boundary is
50 confined by mechanism rather than by assertion, and what it contaminates is read from the pipeline’s
51 own dataflow. We are not aware of prior work scoring an agent’s own boundary trust labels against
52 faults injected into its own pipeline with known ground truth.

53 The paper’s second contribution is methodological and, we think, the more transferable one. Five
54 times during this work a measurement produced a confident number that was an artifact of how we
55 had built the experiment rather than a property of the system: a static-code arm that scored 75 %
56 because our faults were source mutations and collapsed to 0 % once they moved into recorded data;
57 two models that returned a fixed answer regardless of input while appearing to score respectably;
58 and a selection result we reported as significant on a third of its eventual sample and had to retract.
59 Each would have survived into a published table. We report them alongside the results because a
60 benchmark for verifying agents that cannot catch its own defects has no standing to measure anyone
61 else’s. Extended related work appears in Appendix D.

62 **2 The system under test**

63 The pipeline is a marketing research agent. A model authors Python that discovers market demand
64 from public sources, a scanner executes it and captures the values crossing each function boundary,
65 a reviewing session judges each boundary, and a repairing session returns a change confined to one
66 boundary.

67 Three properties matter. The generated pipeline declares its own review boundaries as named func-
68 tions, and the review and repair machinery anchors to those declarations. Stage inputs and outputs
69 pass by argument and return value rather than shared state, which makes a stage replayable against
70 its recorded inputs. And a run records a network cassette, so a pipeline that fetches third-party
71 sources replays against exactly the responses the original run saw. The cassette is what makes this
72 work deterministic, and later what lets us move faults out of the source entirely.

73 **3 What the published comparison can and cannot settle**

74 The published comparison ran four arms over the same frozen pipeline, input and recorded corpus,
75 differing in what the reviewer sees: stage metadata, accumulated history, the complete graph, or
76 a selected part of it (table in Appendix A). It is carefully controlled: the same frozen pipeline,
77 input and recorded corpus across arms, fresh sessions, no cross-arm visibility. Three properties
78 nonetheless prevent it settling which regime is better, and they determine what this paper had to
79 build.

80 Sample size is the first. One run per arm and three repairs within each makes every quantity a count
81 out of three or a single ordinal, and the recommended arm ties the accumulated-history baseline on
82 effective repairs. Second, the arms were not adjudicated identically: the archived record states the
83 first common full-graph judge exceeded the model’s 1,048,576 character input limit on the history
84 branch, and those runs were rejudged with isolated graph-selected judges, a failure that also removed
85 that arm’s timing. Third, and most usefully, the function selecting which boundary to repair differs
86 across arms. The graph arms filter flagged units to causal roots using upstream identifiers; the
87 baselines rotate with no causal step. The comparison is presented as one of context regimes but the

88 arms also differ in how they choose a target. Rather than control that away we make it the treatment,
89 and it produces the paper’s largest effect.

90 **4 Injecting a fault whose location is known**

91 The missing ingredient is an answer. If we know which boundary is wrong, a trust signal stops being
92 something the system relies on and becomes something we can score.

93 We obtain that answer by injecting the fault ourselves. A fault is a small transformation of one
94 boundary function’s source. It is applied by locating the function, rewriting its abstract syntax tree,
95 and splicing the result back through the same scope validator a repair must pass. That last detail is
96 what makes the ground truth trustworthy rather than asserted: an injected fault is confined to one
97 boundary by the same mechanism that confines a repair, so if the confinement is broken the injection
98 fails rather than silently spreading.

99 Contamination is read from the pipeline rather than declared by us. A boundary is downstream of the
100 injected one when the injected function’s result reaches its arguments, either directly as a nested call
101 or through a module-level name bound to that result. Ground truth is therefore a pair: the injected
102 boundary, and the ordered list of boundaries its output reaches.

103 Two guards keep the corpus honest. An injection that leaves the source unchanged is refused, be-
104 cause a fault nothing can detect would enter the corpus and depress every signal’s score equally,
105 making every arm look worse without distinguishing them. And a test asserts that every fault in the
106 catalogue still changes something on a representative pipeline; when we added it, it caught one that
107 did not.

108 **4.1 The fault catalogue**

109 The catalogue separates faults observed in this system’s own operational logs from synthetic ones,
110 because the two carry different weight. Three are observed. A field disappears from a stage’s output
111 after upstream source reads failed, which this system’s logs record alongside HTTP 429, 403 and 500
112 responses from four source registries. A collected sequence is shortened while the stage still reports
113 success. An identifier is replaced by a plausible placeholder, which is what a schema-bound worker
114 actually returned before a reviewing model discarded its output. Two are synthetic: a comparison
115 operator is inverted, and a ranking is reversed while every value in it stays present and valid.

116 An invariant here is an acceptance criterion attached to a boundary and checked deterministically
117 against the value it produced; a concrete example is given in Appendix C.

118 **4.2 What a trust signal is scored on**

119 A signal labels every boundary trusted or suspect. Scoring compares those labels with the injected
120 fault and reports four things.

121 *False trust* is the injected boundary being labelled trusted. This is the safety failure, and a signal that
122 labels everything trusted scores perfectly on cost and fails completely here.

123 *False suspicion* is a boundary that is neither the injected one nor contaminated by it being flagged.
124 This is the cost, paid in redone work.

125 *Localisation error* is signed, and the sign carries the finding. The resume point is the earliest suspect
126 boundary, which is where a run acting on these labels would restart. A negative error means it
127 restarts upstream of the fault and redoes sound work. A positive error means it restarts past the
128 fault and keeps output the fault has already contaminated. Averaging the two would hide the second
129 failure inside the first, so we report them separately.

130 *Shipped contaminated output* records whether the positive case occurred.

131 **4.3 Arms**

132 Four signals are in scope. Structural schema validation checks a stage’s output against its declared
133 JSON schema. Content invariants check criteria the boundary declared in advance: required fields,

Shape	Declaration order	Contested	Causal root	Source order
chain-8	dataflow	16	100 %	100 %
chain-8	scrambled	16	100 %	38 %
fan-6	scrambled	2	100 %	0 %
diamond-4	scrambled	14	100 %	14 %
chain-3	scrambled	6	100 %	0 %
all	dataflow	38	100 %	100 %
all	scrambled	38	100 %	21 %

Table 1: Repair-target selection across four pipeline shapes. The causal-root selector is exact in every cell. The baseline matches it only when declaration order already encodes execution order.

134 minimum counts, value patterns and ordering. Both are deterministic and cost nothing per fault,
 135 which is what allows the corpus to be large. The third arm is the reviewing model itself, which is
 136 the only arm that costs tokens and the only one whose labels vary between runs on the same input.

137 5 Results

138 The results divide along the paper’s claim. Sections 5.2 to 5.3 concern computing over the graph’s
 139 structure, where it pays. The final part concerns showing the graph to a model, where it does not.

140 5.1 Detection: what declared signals catch

141 A first corpus crashed on seventeen of twenty-eight faults, so execution is itself a strong free detec-
 142 tor; but a fault that halts the run never reaches a trust decision, and real generated pipelines read
 143 inputs through accessors with defaults. Everything below concerns faults such a pipeline survives
 144 silently: 104 cases across two pipelines, the cross product of five fault classes, the boundaries and
 145 the applicable parameters, most from classes observed in this system’s own logs.

146 Structural schema validation leaves 78 % undetected, declared invariants 27 %, and a model judge
 147 18 %, none of them raising a single false suspicion. The intervals for the latter two overlap and an
 148 exact McNemar test does not separate them ($p = 0.16$), so we make no claim that either is better;
 149 what the sample supports is that both are significantly worse than their union, which leaves 7 %
 150 ($p < 0.0001$ and $p = 0.0005$). The failures are disjoint. Invariants catch twelve faults the judge
 151 misses, every one a matter of counting or shape; the judge catches twenty-one the invariants miss,
 152 eighteen of them substituted identifiers including every plausible one that defeated the pattern check.
 153 Schema validation contributes nothing either does not, detecting only the fault class that changes a
 154 record’s shape.

155 5.2 What the graph is actually for

156 Detection tells a run something is wrong. It does not say what to repair. When several boundaries
 157 are flagged, something must choose, and that choice is where the graph does work no cheaper signal
 158 can. We measure a selector that walks recorded dataflow to the causal root against one that walks
 159 the declared boundary list in order.

160 The comparison is only meaningful when those two orders differ. In a pipeline whose functions are
 161 declared in the order they run, the causal root of any flagged set is simply the first of them, and
 162 the selectors cannot disagree. Models writing code do not reliably declare in execution order, so
 163 we test both conditions across four shapes: a deep chain, a wide fan-out, a diamond that splits and
 164 reconverges, and a short chain.

165 The causal-root selector is exact in all 38 contested cases of every shape and both orderings. The
 166 baseline matches it perfectly when declaration order happens to encode execution order and falls to
 167 21 % when it does not. This gives the graph’s value a precise form: *it is worth exactly the extent to*
 168 *which declaration order diverges from execution order*. A team whose pipelines are written in order
 169 gains nothing here. A team whose pipelines are model-authored gains the difference between 100

Signal	Missed	Rate	95 % CI
schema validation	81/104	78 %	[69, 85]
declared invariants	28/104	27 %	[19, 36]
model judge (3B)	19/104	18 %	[12, 27]
invariants + judge	7/104	7 %	[3, 13]

Table 2: Faults left undetected over 104 cases, with Wilson intervals. The intervals for invariants and the judge overlap; the pair is separated from both.

and 21 per cent, and cannot know in advance which case they are in, which is itself an argument for computing the relation rather than assuming it.

5.3 The model judge, and where each signal fails

We expected the language-model judge to be the weak arm. It was not, and reporting what happened instead is more useful than the result we set out to find.

The judge is a 3B model run locally, asked about one boundary at a time and given that stage’s declared purpose in prose. Across both corpora, 104 silent faults with three repeats each, it never once disagreed with itself. Every one of the 104 cases came back unanimous over 312 calls, with no parse failures.

A false-trust rate alone would mean nothing, because a judge answering suspect indiscriminately scores perfectly while being useless. We therefore asked the same judge about boundaries known to be sound: every boundary of the clean unfaulted pipeline, and for each case a boundary strictly upstream of the fault, which cannot be contaminated. Across 102 such calls it returned trusted every time, so its false-suspicion rate matches both deterministic signals at zero and the comparison stands.

5.4 What the sample supports, and what it does not

The ranking in Table 2 invites a conclusion the data does not carry, so we tested it on the paired outcomes. An exact McNemar test does not separate the judge from declared invariants (12 against 21 discordant, $p = 0.16$), and their confidence intervals overlap. We make no claim that either is the better signal, and the sample is large enough that this is a finding rather than a shrug: two signals of quite different kinds, one a compiled check and one a 3B model, are not distinguishable by error rate.

The combination is a different matter. It improves on invariants alone (0 against 21, $p < 0.0001$) and on the judge alone (0 against 12, $p = 0.0005$). Both single signals are significantly worse than using both; neither is significantly worse than the other. Schema validation is separated from all three at $p < 0.0001$, and adding it to any pair changes nothing, because it catches nothing they do not.

The result is therefore about complementarity rather than ranking. Invariants catch twelve faults the judge misses, and every one is a matter of counting or shape: fields removed from a record, sequences silently shortened. The judge catches twenty-one the invariants miss, and eighteen of those are substituted identifiers, including every plausible one that defeated the pattern check. Each signal is blind close to where the other sees, and it is that disjointness the evidence establishes.

It would be wrong to read this as a model outperforming a static check. The judge was given the stage’s declared purpose, for instance that a retrieval stage must fetch every source in every declared class and report what it retrieved. That is an acceptance criterion. It is the same kind of statement the invariant encodes, written in prose and checked by a model rather than compiled into a count and a regular expression.

So the comparison was never between having criteria and not having them. It was between two encodings of the same criteria, and the encodings fail differently. A count is exact about quantity and silent about meaning. A sentence carries meaning and is vague about quantity. That is why the union is so much stronger than either part, and it is a reason to declare criteria at a boundary at all rather than a reason to prefer one checker.

Regime	Accuracy	Context	Note
stage purposes only	15 %	1,343	no captured values
size-matched random slice	35 %	1,402	control
graph-selected region	40 %	1,466	$p = 0.69$ vs random
selected plus expansion	46 %	1,466	the deployed method
complete current run	50 %	1,913	
accumulated history, d=64	40 %	129,246	

Table 3: Localisation under six context regimes. Selection is not separable from a random slice of the same size, including in the form the system actually deploys; accumulated history costs two orders of magnitude more for less.

Arm	Faults	Clean	Mean chars	Expansions
collapsed, expansion on request	5/6	4/4	841	9
everything, flattened	5/6	4/4	922	0
random subset, budget matched	4/6	3/4	831	0

Table 4: The deployed protocol against sending everything and against a size-matched random subset. It matches the former on nine per cent less context and, unlike every fixed-slice arm we tested, does not tie the latter.

5.5 Showing the graph to a model does not pay

The use an execution graph is most often justified by its presentation: give the agent a relevant slice rather than everything or nothing. Four regimes were asked the same question over a branching corpus, name the faulty stage, with the injected fault as the answer.

Captured values matter enormously, 15 % against 40 to 50 for any regime showing what the code produced, which is the strongest evidence here that execution records are the right substrate. Selecting among them does not help: the graph-selected region is not separable from a size-matched random slice ($p = 0.69$), and adding expansion on request, which is the method as deployed rather than a weakened stand-in, reaches 46 % against 50 % for simply showing everything. Accumulated history degrades from 52 % at depth four or less to 40 % at depth sixteen or more while its context grows to 129,246 characters against 1,913. The mechanism is visible in the failures: shown captured values, a model names a downstream stage in 83 % of cases, and selection acts on the prompt rather than on the answer, so a model handed a selected region still blames the most downstream thing in it.

5.6 A protocol that withholds and is asked

The arms above hand the reviewer a slice chosen in advance. The system we study does something different, and the distinction turns out to matter. Its initial package carries nodes at exactly the boundary’s function-stack level, names the nested helpers without their contents, and permits the reviewer to request one direct child at a time for at most three rounds. Evaluating that needs a pipeline whose boundaries call helpers, so we built one and planted faults at three depths: visible at the boundary, inside a helper, and inside a sub-helper.

This is the only selection arm in the paper that separates from a size-matched random control, and it forces a narrower statement of the result above. A selected region handed over in advance is not better than a random one of the same size. A protocol that withholds nested detail and lets the reviewer pull what it needs is a different mechanism, and on this evidence it works: the same accuracy as sending everything, for less, and better than random at equal cost. What we can say is that the benefit lies in the interaction rather than in the choice of slice, which is not what we expected and not what the fixed-slice arms could have shown.

Two details qualify it. Faults planted two levels deep were caught with zero and one expansions, because a deep fault usually surfaces in the boundary’s own output, so the reviewer rarely has to drill to where the fault lives. And the single miss is a reversed ranking judged clean after an expansion was spent, which is precisely the fault an ordering scar catches deterministically in Section 5.9. With

242 ten cases per arm this is directional rather than sized, and we implemented the collapsed-helper half
243 of the protocol but not its artifact inspection.

244 5.7 External validity: three recovered runs of the real system

245 Everything above runs on pipelines we authored, which is the weakest thing about it. Three records
246 of the real system, deleted in a cleanup commit but recoverable from its git history, supply the
247 external check.

248 A four-arm run of 29 July 2026, on the real model-authored pipeline with a frontier model, reports
249 that graph-selected context matched the full graph’s issue coverage with 46 % fewer input tokens on
250 the initial run and 47 % on the latest, while accumulated flat history grew from 71,470 to 303,757
251 characters and became 29 % more expensive than the selected region. A second run of 30 July
252 shows the same shape at 31,369 tokens for stage metadata against 109,590 for accumulated history,
253 all arms finding the same three issue functions. This is the efficiency half of our claim, measured on
254 the system itself.

255 The same 29 July record contains one further observation we regard as the most important sentence
256 in either dataset: accumulated history *retained a previously repaired boundary as an open issue after*
257 *every other arm treated it as resolved*, which its authors attribute to stale prior versions remaining
258 salient in flat history. Our synthetic history sweep never produced this, and the recovered run shows
259 why: our accumulated history was clean prior iterations, which hand the model a helpful baseline to
260 diff against, whereas real history contains superseded versions of boundaries that have since changed.
261 Staleness, not volume, is the active ingredient. Section 5.8 tests that as an intervention.

262 5.8 Staleness, briefly

263 Accumulated history harms a model judge, and what it does depends on the model. A superseded
264 faulty run in context halves one judge’s detection of genuine new faults; another judge cannot recog-
265 nise a clean run without one, inventing a fault in sixteen of twenty clean cases and naming the final
266 stage every time; the deployed system’s own record shows a third model resurrecting a resolved
267 issue from flat history. Neither of ours re-flagged the repaired boundary, which was the predicted
268 failure. The stable conclusion is a dependence rather than a direction: history content modulates
269 judge behaviour in model-specific ways that no prompt here controlled, while deterministic signals
270 are unaffected by context by construction. Full cells in Appendix E; the detection drop does not
271 survive the multiplicity correction of Section ??.

272 5.9 Checks placed where faults enter

273 Every signal so far was written in advance by an author deciding what to assert. The alternative is
274 to learn a check from a fault once it has been caught, place it on the boundary where it entered, and
275 pay for that fault only once.

276 We adopt the term *scar* from prior work on coupling kernels, where a scar is a low-rank invariant
277 subspace that lets a query be *read* exactly rather than searched over an intractably large state space
278 [9], and from its use in agent goal induction, where a goal is treated as a low-rank commitment set
279 read in a number of probes proportional to its rank [8]. The move transfers directly. A caught fault
280 commits a small set of properties; rather than re-deriving them on every subsequent run, they are
281 read off once and pinned to the boundary that produced them. What is new here is the application
282 to trust decisions and the measurement of what it buys, which separates the two halves of this paper
283 more sharply than anything else.

284 A scar is minted from a population of healthy observations of one boundary and one faulty observa-
285 tion of the same boundary, keeping only structural properties that held across the whole population
286 and broke in the faulty one: a field present throughout and now absent, a count below its healthy
287 floor, a list that stopped being homogeneous, an ordering that held and no longer does. A property
288 expressible as no such check is refused rather than stored as vague suspicion, because the determin-
289 istic signals’ zero false-suspicion rate is the thing that makes them worth having.

290 Table 5 contains the result we consider most important in this paper. Hand-written contracts detect
291 26 of 90 faults and localise *none* of them. A stage that faithfully passes on a bad input satisfies every

Signal	Detected	Localised	Misplaced
declared contracts	26/90	0/90	26/90
learned scars	60/90	60/90	0/90

Table 5: Ninety data faults. Localisation is whether the flagged boundary is the one the fault entered. Declared contracts never localise: every detection fires on an aggregating stage downstream of the cause.

criterion anyone thought to write about its own behaviour, so the checks fire on the aggregating stages downstream and the suspect set sits entirely past the cause. This is the same failure the model judges showed, in a signal with no model in it, which tells us the downstream-blame bias is not a property of language models but of where checks are placed. Scars localise every fault they detect, because a scar is placed where the fault was observed to enter rather than where an author’s attention happened to fall.

Two further results bound the method. Population size does not affect detection at all, identical at one, four and eight healthy observations; what it buys is false suspicion, falling from 21 of 56 boundary-checks at $n = 1$ to zero at $n = 8$. A single healthy observation cannot separate an invariant from that run’s value, so scars minted from one pair fire on honest variation, and only a population can tell the difference. Second, a vocabulary check is minted only when the population shows the domain closed: identifiers drawn from a fixed registry stop admitting new values, dates and free text do not, and a vocabulary check over an open domain is guaranteed to fire on an honest run eventually. Enforcing that costs detection, 73 falling to 60, and takes false suspicion on held-out healthy runs the scars never saw from 3 of 28 to zero. We regard that trade as the correct one, and the refusal of faults visible only in an open domain as a feature of the design.

We also implemented the two graph computations this suggests, and both are negative. Forward propagation of contamination reclassifies nothing, because the declared checks fire downstream of the fault and there is nothing to propagate forward from. Backward attribution narrows to seven boundaries of seven, because in a pipeline that funnels through a merge every branch is an ancestor of the first suspect. Both failures have one cause: exoneration requires a check that the fault *would have violated*, not merely a check that passed, and a hand-written contract does not supply one where a scar does.

6 Defects, in the system and in the measurement

6.1 In the trust machinery

Four defects, detailed in Appendix B: the repair-scope validator silently skipped enforcement whenever a declared boundary name failed to resolve, a branch the authoring model controls and that the entire workflow test suite unknowingly ran through; the module producing the published numbers had no tests; resource limits around generated code were inactive by default; and the real home directory, with credentials, reached model-written code that has network access outside replay [4].

6.2 In our own measurement

A benchmark built to verify agents has no standing to measure anyone else if it cannot catch its own defects, and five times a confident number here turned out to be an artifact of how we built the experiment.

A control arm shown the source text of the implicated region scored 75 %, appearing to refute the premise that captured values matter; our faults were source mutations, so it was reading the fault rather than diagnosing it, and moving faults into recorded responses with the source byte-identical across all 48 cases dropped it to 0 %. A 3B judge scored 0 % on seven-way localisation while returning valid stage names, answering one of two regardless of input; a 30B model later returned `transform_3` for all 48 cases at every chain length from five to seventeen stages, producing a flat apparent degradation with horizon that was a constant. Both look like results until the answer distribution is examined, and neither can exhibit an effect of context because neither depends on context. We reported that graph-selected context beat a size-matched random slice, 40 against 28 %, on a

third of the eventual sample; at full sample the difference vanished ($p = 0.69$), and the claim was retracted, having been made in the same paragraph as a criticism of single-run reporting. Because data-level faults land on the stage that fetches the corrupted response, ground truth concentrated on three of seven boundaries, making the correct chance baseline 42 % rather than 14 %, and an arm shown no captured values scored exactly that by always naming the most common answer. Finally, a dump cap truncated retrieval evidence before the judge saw it, so faults on later sources were invisible and the judge blamed downstream aggregates. Each would have survived into a table, and we take the frequency as evidence that scoring against constructed ground truth is worth as much for what it invalidates as for what it establishes.

7 Limitations

Three judges appear here, a 3B, a 30B mixture and one frontier model, with large and non-uniform capability differences, so no rate should be read as a property of models in general; and the judge was never asked to choose among boundaries the way the selector does, so the selection and detection results are not one story.

Faults are injected rather than arising from a model, and model-authored faults would differ systematically from ours; scars inherit this doubly, learning only differences observable at the boundary where the fault entered, and evaluated on one fault taxonomy. Our contamination model follows module-level dataflow, so state passed through a file, a global or a side effect defeats it, which makes it sound for conforming pipelines and unsound for the ones most likely broken. Whether the unresolvable-target defect fired during the published run cannot be determined from the repository, and the recovered records corroborate the efficiency claim without bearing on detection rates. Finally, no pipeline exceeds seventeen stages, so nothing measures the true long-horizon regime [2].

References

- [1] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.
- [2] Mingguang Chen, Licheng Wang, and Bo Qu. The horizon gap: Planning, memory, execution, training, and evaluation for long-horizon LLM agents. *arXiv preprint arXiv:2608.06663*, 2026.
- [3] Juliana Freire, David Koop, Emanuele Santos, and Claudio Silva. Provenance for computational tasks: A survey. *Computing in Science & Engineering*, 10(3):11–21, 2008.
- [4] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- [5] Lukas Helff, Quentin Delfosse, David Steinmann, and Ruben Härle. LLMs gaming verifiers: RLVR can lead to reward hacking. *arXiv preprint arXiv:2604.15149*, 2026.
- [6] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.
- [7] Justin D. Norman, Michael U. Rivera, and D. Alex Hughes. Reliability without validity: A systematic, large-scale evaluation of LLM-as-a-judge models across agreement, consistency, and bias. *arXiv preprint arXiv:2606.19544*, 2026.
- [8] Fabio Rovai. CIVEX: Causal intervention verification for language agents. *arXiv preprint arXiv:2605.09168*, 2026.
- [9] Fabio Rovai. WorldKernel: A world model is the coupling kernel of admissible possible worlds. *arXiv preprint arXiv:2606.10934*, 2026.
- [10] Kunvar Thaman. Reward hacking benchmark: Measuring exploits in LLM agents with tool use. *arXiv preprint arXiv:2605.02964*, 2026.

- [11] Yiqi Wang, Jiaqi Zhang, Taotao Cai, and Zirui Liu. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. *arXiv preprint arXiv:2606.04990*, 2026.
- [12] Koki Wataoka, Tsubasa Takahashi, and Ryokan Ri. Self-preference bias in LLM-as-a-judge. *arXiv preprint arXiv:2410.21819*, 2024.
- [13] Yu Zhuang, Kefei Chen, Yitong Duan, and Shuxin Zheng. AgentRewind: Recoverable execution for long-horizon LLM agents. *arXiv preprint arXiv:2608.14380*, 2026.

Artifact availability

The benchmark, every corpus, all result files and the analysis scripts accompany this submission as an anonymized artifact and will be released publicly on acceptance; the recovered records of Section 5.7 are identified by the commit hash of the repository they were recovered from.

A The published comparison

Arm	Repairs	Effective	Input tokens	Unresolved boundaries
semantic_only	3	0	568,715	4
history_full	3	1	1,824,533	4
etiq_full	3	0	1,331,686	3
etiq_selected	3	1	1,000,797	2

Table 6: The published comparison, one run per arm, 30 July 2026, taken from the archived machine-readable summary. Its own record notes that “no arm reached final trust”.

B Defects in the trust machinery, in detail

The validator confining a repair to one boundary returned early whenever the target could not be resolved to a function, so in that branch a repair could rewrite the whole target file. The branch is reached whenever a declared boundary name does not resolve in the generated source, and the authoring model writes both, so it is reachable by ordinary error and steerable on purpose; the harness splices over the same span, so an unresolvable target produced a whole-file substitution the validator accepted. It was untested, because the scanner fixture reported a call stack inside functions the authoring fixture never defined, so no target ever resolved and every workflow test of repair ran through the unenforced branch. Separately, the module producing the published numbers was imported by no test, resource limits around generated code defaulted to inactive, and the home directory reached the generated pipeline, which has network access outside replay mode; captured third-party text now reaches the reviewer inside a digest-keyed fence it cannot close [4].

C A declared invariant, concretely

The retrieval stage of the branching pipeline carries: `retrieve_registries` must return `source_ids` and `source_class`; `source_ids` must hold at least three entries; every entry must match `^[a-z]_[a-z]+`. Three clauses catching a disappeared field, a silently shortened retrieval, and a malformed identifier; what none can catch is an identifier that is correctly shaped and simply wrong, which is measured rather than assumed in the body.

D Extended related work

Empirical work on why multi-agent systems fail supplies the closest framing for our problem: Cemri et al. build a taxonomy from more than two hundred annotated traces and make task verification one of its three top-level categories [1], though their failures are annotated by human experts rather than

417 detected by the system at runtime; Chen et al. survey the same territory as a gap between what long-
 418 horizon agents are asked to do and what their planning, memory and evaluation machinery supports
 419 [2]. The assumption our benchmark tests is that a language model supplies that verification, and
 420 there is reason for caution: judges favour text more familiar to themselves [12], which matters
 421 because reviewer and reviewed here share a model family; a judge’s self-agreement can be high
 422 while its agreement with the truth is not [7], the distinction our localisation metric operationalises;
 423 verifiers checking only what they can express admit confident false positives [5]; and multi-step
 424 agents exploit naturalistic chances to tamper with what evaluates them [10]. Provenance-based trust
 425 for agents [11] and recoverable execution from recorded checkpoints [13] are the constructive lines
 426 this work measures rather than proposes.

427 Our method borrows its central move from mutation testing, a defect of known location measuring
 428 whether a suite detects it, including the equivalent-mutant problem we meet as an injection that
 429 changes nothing [6]. The substrate is execution provenance [3], whose distinction between a record
 430 of what a computation did and a claim about what it meant locates this paper’s difficulty: the graph is
 431 a record, the trust label is a claim, and only the first is checkable. The fencing of Section 6 addresses
 432 indirect prompt injection through fetched content [4], a designed-in exposure here since the pipeline
 433 must retrieve third-party sources at runtime.

434 E Staleness intervention, full results

435 We crossed two factors, twenty cases per cell on each of two judges: the history shown (clean prior
 436 iterations, or a superseded run carrying a fault since repaired) and the current run (clean, or carrying
 437 a new fault at a different boundary), asking for the faulty stage of the current run or none.

History	Current run	Judge A	Judge B
clean	clean	18/20	4/20
stale	clean	17/20	13/20
clean	new fault	18/20	19/20
stale	new fault	11/20	19/20

Table 7: Correct decisions under the staleness intervention. Each judge fails in a different place: A dismisses genuine new faults when a stale prior is present, B cannot recognise a clean run unless one is.

438 Neither judge re-flagged the repaired boundary, which was the predicted failure; that happened
 439 once in 160 stale-history calls. Judge A, handed a superseded faulty prior, misses genuine new
 440 faults, 18/20 falling to 11/20 (Fisher exact, $p = 0.03$ uncorrected, suggestive after the multiplicity
 441 correction of Section ??); the prior appears to act as a contrast anchor, and a current run that looks
 442 better than the old one reads as fine. Judge B shows the opposite defect: it detects the new fault
 443 regardless of history, 19/20 in both cells, but cannot recognise a clean run, inventing a fault in 16
 444 of 20 clean-history cases and naming the final stage in every one, which is the downstream-blame
 445 bias with nothing to blame; a superseded faulty prior partially restores its ability to answer clean,
 446 13/20 against 4/20. The deployed system’s own record supplies a third signature, a frontier model
 447 resurrecting a resolved issue from flat history.

448 Three judges, three failure shapes from one manipulation. The stable conclusion is a dependence
 449 rather than a direction: the content of accumulated history modulates a model judge’s trust decisions
 450 in model-specific ways that no prompt here controlled, while deterministic signals are unaffected by
 451 context by construction.